

Mini-projet 10 — Les décorateurs (le capot ouvert)

Module C · puissance · Notions : **fonctions** = **objets** · **fonction imbriquée** · **décorateur** · **sucré** @ · **functools.wraps** · Durée : 2 h · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Au MP09 tu as **utilisé** des décorateurs (`@classmethod` ...). Maintenant tu vas en **fabriquer**. Un décorateur, c'est une fonction qui en **emballe** une autre pour lui ajouter un comportement (mesurer son temps, la relancer si elle échoue, journaliser ses appels) **sans toucher à son code**. Dans ta boîte à outils, tu voudras chronométrer un scan, relancer une connexion réseau qui a raté, logger chaque appel de plugin. C'est exactement ça.

1. La clé : en Python, une fonction est un objet

Comme un nombre ou une liste, une fonction peut être rangée dans une variable, passée en argument, ou renvoyée. C'est ce qui rend les décorateurs possibles.

```
def saluer():
    return "salut"

f = saluer          # on range la fonction dans f (SANS parenthèses !)
print(f())          # "salut" — f EST saluer
```

Nuance capitale : `saluer` (sans `()`) = la fonction elle-même, comme objet. `saluer()` (avec `()`) = **appeler** la fonction et récupérer son résultat. Un décorateur manipule la fonction **sans l'appeler** tout de suite.

2. Une fonction qui en emballe une autre

On peut définir une fonction **dans** une autre, et renvoyer cette fonction interne :

```
def chrono(f):
    # reçoit une fonction f
    def emballage():
        # fonction interne qui "entoure" f
        import time
        debut = time.time()
        resultat = f()          # on appelle la vraie fonction, ICI
        duree = time.time() - debut
        print(f"{f.__name__} a pris {duree:.3f}s")
        return resultat
    return emballage           # on renvoie l'emballage (SANS l'appeler)
```

Utilisation « à la main », sans le `@` pour bien voir le mécanisme :

```
def travail():
    return sum(range(1000000))

travail = chrono(travail)  # on REMPLACE travail par sa version emballée
travail()                  # -> "travail a pris 0.0xx s" puis renvoie le résultat
```

3. Le @ n'est QUE du sucre

Écrire `@chrono` au-dessus d'une fonction fait **exactement** la ligne `travail = chrono(travail)`. Rien de plus.

```
@chrono
def travail():
    ...
    travail = chrono(travail)
```

```
@chrono          # sucre pour : travail = chrono(travail)
def travail():
    return sum(range(1000000))

travail()        # même effet qu'avant
```

4. Emballer N'IMPORTE quelle fonction : *args et **kwargs

Problème : notre `emballage()` n'accepte aucun argument. Mais les vraies fonctions en ont (une cible, un timeout...). Solution : l'emballage ramasse **tous** les arguments avec `*args, **kwargs` (revus au MP02) et les **re passe** tels quels à `f`.

```
def chrono(f):
    def emballage(*args, **kwargs):
        # ramasse TOUS les arguments
        import time
        debut = time.time()
        resultat = f(*args, **kwargs) # les REPASSE à la vraie fonction
        print(f"{f.__name__} a pris {time.time()-debut:.3f}s")
        return resultat
    return emballage
```

Le motif universel : `def emballage(*args, **kwargs): ... f(*args, **kwargs)` . Grâce à ça, `@chrono` marche sur une fonction à 0, 1 ou 10 arguments. C'est le squelette de **tout** décorateur.

5. Détail pro : `functools.wraps`

Sans précaution, la fonction décorée « perd son nom » (elle s'appelle `emballage` partout). `@wraps(f)` corrige ça — mets-le, c'est la bonne pratique :

```
from functools import wraps

def chrono(f):
    @wraps(f) # garde le nom/docstring de f
    def emballage(*args, **kwargs):
        ...
    return emballage
```

Ta mission

Objectif : fabriquer **deux** décorateurs et les appliquer.

1. `@chrono` : mesure et affiche le temps d'exécution de la fonction, puis renvoie son résultat. Doit marcher sur une fonction **avec arguments** (donc `*args, **kwargs`).
2. `@log` : avant l'appel, affiche "appel de <nom> avec args=... kwargs=..." ; après, affiche "<nom> a renvoyé <resultat>" ; et renvoie le résultat.

Applique-les sur une fonction de test, par exemple :

```
@chrono
def addition(a, b):
    return a + b

print(addition(2, 3))
# affiche quelque chose comme :
#   addition a pris 0.000s
#   5

@log
def multiplier(a, b):
    return a * b

multiplier(4, 5)
# affiche :
#   appel de multiplier avec args=(4, 5) kwargs={}
#   multiplier a renvoyé 20
```

Bonus (le plus utile pour le réseau) : un décorateur `@retry` qui relance la fonction jusqu'à 3 fois si elle lève une exception, et ne laisse remonter l'erreur qu'après le 3e échec. (Indice : une boucle `for` dans l'emballage, avec un `try/except` — tout ce que tu sais déjà.)

✓ Critères de réussite

- `@chrono` et `@log` marchent sur une fonction **qui a des arguments** (grâce à `*args, **kwargs`).
- Les deux **renvoient bien le résultat** de la fonction (pas juste afficher).

- Tu utilises `@wraps(f)` dans chacun (bonne pratique).

Si tu bloques — indices progressifs

Étape	Indice
Structure d'un décorateur	Toujours 3 niveaux : <code>def chrono(f):</code> → dedans <code>def emballage(*args, **kwargs):</code> → <code>return emballage</code> .
Appeler la vraie fonction	Dans l'emballage : <code>resultat = f(*args, **kwargs)</code> , et n'oublie pas de <code>return resultat</code> à la fin.
@log — afficher les args	<code>print(f"appel de {f.__name__} avec args={args} kwargs={kwargs}")</code> AVANT l'appel.
@retry (bonus)	<code>for essai in range(3):</code> → <code>try: return f(*args, **kwargs)</code> → <code>except Exception as e: print(...)</code> . Après la boucle, si tout a échoué, <code>raise</code> .

Où ranger : `mini-projets/10-decorateurs/decorateurs.py` .
Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».